

Advanced Deep Learning

TD1 : Photos Organization and Clustering

1 PhotoMap: Organizing an Unlabeled Photo Collection

Imagine that you are given a large collection of personal photographs. The photographs have no labels, no predefined categories, and no particular organization.

Manually browsing and organizing such a collection can quickly become impractical. The objective of this practical session is to build the intelligence behind **PhotoMap**, an application capable of exploring and organizing a collection of photographs using representations learned by pretrained deep learning models.

1.1 The application

A web interface is provided and contains three main views:

- **Collection:** displays the complete collection of photographs;
- **Photo Map:** provides a two-dimensional visual organization of the collection, where photographs with similar representations should tend to appear close to each other;
- **Groups:** automatically organizes the photographs into groups discovered from their representations.

In addition, clicking on any photograph should display its most similar photographs according to the learned representation.

1.2 Dataset

The photo collection used in this practical session is a subset of the **Caltech-256** dataset¹. The original category information has been removed: throughout this practical session, the collection must therefore be considered **unlabeled**.

You must not attempt to recover or use the original Caltech-256 labels. The objective is to explore the structure discovered directly from pretrained image representations.

1.3 Your objective

Your task is **not** to train a deep neural network from scratch. Instead, you will use an existing pretrained representation-learning model as the foundation of the application.

Starting from the provided photographs, you will progressively implement a pipeline of the following form:

¹G. Griffin, A. Holub, and P. Perona, *Caltech-256 Object Category Dataset*, California Institute of Technology, 2007.

$$\text{photographs} \longrightarrow \text{pretrained encoder} \longrightarrow \text{representations} \longrightarrow \begin{cases} \text{2D visualization,} \\ \text{similarity search,} \\ \text{automatic grouping.} \end{cases}$$

The application itself is already provided. Your responsibility is to implement the methods that provide its intelligent behavior.

1.4 Freedom of implementation

There is deliberately no unique solution to this problem.

You are free to choose:

- the pretrained representation model;
- the deep learning framework (*PyTorch*, *TensorFlow/Keras*, or another appropriate framework);
- how representations are extracted and processed;
- the dimensionality-reduction method;
- the similarity measure;
- the clustering method and its hyperparameters.

Your choices must, however, be **justified experimentally and scientifically**. A working application alone is not sufficient: you should be able to explain why the different components of your pipeline are appropriate for the problem.

1.5 Important constraint

The collection must be treated as **unlabeled data**. Any semantic categories that may exist in the dataset must not be used to train or construct the representation.

The objective is precisely to investigate what structure can be recovered from representations learned beforehand, without supervision on the PhotoMap collection.

2 Understanding the Provided Application

Before implementing the representation-learning pipeline, you should first download and explore the provided PhotoMap application.

2.1 Downloading the source code

The source code required for this practical session is available at:

`https://hadifawaz1999.github.io/Advanced-Deep-Learning--3A-Info/TD/TD\_01\_Photos\_Organization/TD\_01\_Photos\_Organization.zip`

Download and extract the archive before continuing.

The archive contains the PhotoMap web application, a mock pipeline used to demonstrate its behavior, and the photo collection that will be used throughout this practical session.

2.2 Provided files

After extracting the archive, you should obtain the following structure:

```
TD_01_Photos_Organization/
|
|-- app.py
|-- random_pipeline.py
|-- requirements.txt
|
|-- data/
|   |-- mock_photos/
|   |-- photos/
|
|-- static/
|   |-- app.js
|   |-- style.css
|
|-- templates/
|   |-- index.html
```

The web interface is already implemented. You are therefore **not expected to modify the HTML, CSS, or JavaScript code**.

The file `app.py` implements the connection between the web interface and the image-processing pipeline.

The file `requirements.txt` contains the minimal dependencies required to run the provided application. You are free to install and add any additional dependencies required by your own implementation.

2.3 Running the application

We recommend creating a dedicated Python environment before installing the dependencies.

Install the dependencies of the provided application with

```
pip install -r requirements.txt
```

Then run PhotoMap with

```
python app.py
```

Open the address displayed in the terminal in your web browser and explore the different functionalities of the application.

You should be able to:

- browse the complete photo collection;
- visualize photographs on the **Photo Map**;
- inspect the automatically generated **Groups**;
- click on a photograph to display its nearest photographs.

At this stage, these functionalities do not rely on any meaningful representation of the photographs.

2.4 The random pipeline

To allow you to explore the application before implementing any machine learning component, a complete mock pipeline is provided in

`random_pipeline.py`

This pipeline generates a small collection of artificial photographs and simulates the different outputs expected by the application.

In particular, the positions in the Photo Map, the groups of photographs, and the nearest photographs do **not** result from a deep learning model or from learned image representations.

They are only provided to illustrate the expected behavior of PhotoMap and the interface between the application and its backend.

The mock photographs are stored separately from the real collection in

`data/mock_photos/`

while the collection that you will organize during this practical session is located in

`data/photos/`

2.5 Your pipeline

Your objective during this practical session is to progressively replace the random pipeline with your own implementation.

Create a new file

`pipeline.py`

which will contain your implementation.

From the point of view of the application, your pipeline must provide the following main operations:

```
get_photos(...)
project_photos(...)
find_neighbours(...)
discover_groups(...)
```

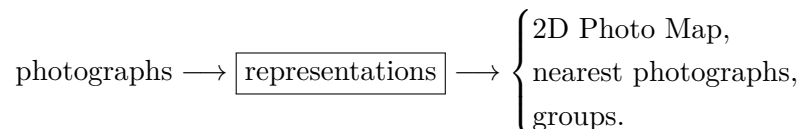
These operations correspond to the different functionalities exposed by PhotoMap.

Internally, however, your implementation will rely on a common representation of the photographs. A typical organization could therefore include an additional operation such as

```
extract_embeddings(...)
```

which computes the representation vectors subsequently used by the other components.

Conceptually, the complete pipeline will have the following structure:



You will implement these components progressively throughout the practical session.

2.6 Connecting your implementation

The provided `app.py` initially uses the mock pipeline and the artificial photo collection. At the beginning of the practical session, you should therefore find the following import:

```
from random_pipeline import (
    discover_groups,
    find_neighbours,
    get_photos,
    project_photos,
)
```

and the following definition of the photo directory:

```
PHOTO_DIR = Path("data/mock_photos")
```

This configuration allows you to run and explore PhotoMap immediately, without having implemented any part of the machine learning pipeline.

Once you start using your own implementation, replace the import with

```
from pipeline import (
    discover_groups,
    find_neighbours,
    get_photos,
    project_photos,
)
```

You must also replace the mock photo directory with the directory containing the actual photo collection:

```
PHOTO_DIR = Path("data/photos")
```

Therefore, the application initially uses

`random_pipeline.py + data/mock_photos/`

and your final implementation should use

`pipeline.py + data/photos/`.

The rest of `app.py` should not need to be modified.

Your objective is that, at the end of the practical session, all the information displayed by PhotoMap is computed from the actual photographs and your own representation-learning pipeline.

2.7 Backend freedom

There is deliberately no imposed architecture for your `pipeline.py`.

You are free to:

- use the deep learning framework of your choice;

- create additional Python files or classes;
- introduce additional functions;
- install additional dependencies;
- cache intermediate results such as image representations;
- organize your backend in any way you consider appropriate.

You may also update your `requirements.txt` to include the dependencies required by your solution. However, the interface between your pipeline and `app.py` must remain compatible with the provided application.

In other words, you are free to change **how** the results are computed, but the information returned to the application must follow the expected format.

2.8 Important

The file `random_pipeline.py` is a development and demonstration tool. It is **not** a solution to the practical session.

You should progressively replace its artificial behavior with results computed from the actual photographs and their learned representations.

At the end of the practical session, the following transformation should have taken place:

$$\boxed{\text{random_pipeline.py} + \text{data/mock_photos/}} \longrightarrow \boxed{\text{pipeline.py} + \text{data/photos/}}.$$

3 Representing the Photo Collection

The first step towards replacing the random behavior of PhotoMap is to construct a meaningful numerical representation of each photograph.

Given a photograph x_i , a pretrained encoder f_θ produces a representation

$$\mathbf{z}_i = f_\theta(x_i), \quad \mathbf{z}_i \in \mathbb{R}^d,$$

where d is the dimensionality of the representation.

For a collection containing N photographs, the objective is therefore to construct the representation matrix

$$Z = \begin{bmatrix} \mathbf{z}_1^\top \\ \mathbf{z}_2^\top \\ \vdots \\ \mathbf{z}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

This same representation matrix will later be used for visualization, similarity search, and automatic organization of the collection.

3.1 Choosing a pretrained encoder

You will **not train a representation model** during this practical session. Instead, choose an existing pretrained model and use it as an image encoder.

You are free to choose both the model and the deep learning framework.

The following models provide possible starting points.

Keras / TensorFlow

- **DINOv2** — self-supervised representation learning based on self-distillation.

https://keras.io/keras_hub/api/models/dinov2/

- **ResNet** — a supervised ImageNet model that can be used as a baseline against self-supervised representations.

<https://keras.io/api/applications/resnet/>

PyTorch

- **DINOv2** — self-supervised representation learning based on self-distillation.

<https://github.com/facebookresearch/dinov2>

- **DINO** — self-supervised learning using a teacher–student architecture.

<https://github.com/facebookresearch/dino>

- **Barlow Twins** — self-supervised representation learning based on redundancy reduction.

<https://github.com/facebookresearch/barlowtwins>

- **ResNet** — pretrained supervised ImageNet models available through Torchvision.

<https://pytorch.org/vision/stable/models/resnet.html>

These models are only suggestions. You may use another pretrained model if you consider it appropriate.

In particular, models studied during the course, such as contrastive, self-distillation, or other self-supervised approaches, are valid choices provided that pretrained weights are available.

3.2 Task 1 — Extract the representations

Implement the first component of your `pipeline.py`.

Your implementation should take the photographs contained in

`data/photos/`

and compute one representation vector for each photograph.

Conceptually, your implementation could expose an internal operation such as

```
embeddings = extract_embeddings(photos)
```

where

```
embeddings.shape == (N, d)
```

for N photographs and a representation dimension d .

The exact implementation is left to you.

Depending on your selected model and framework, you may need to consider:

- loading the pretrained weights;
- applying the preprocessing expected by the model;
- resizing and normalizing the input images;
- disabling gradient computation during inference;
- processing several photographs in batches;
- selecting the appropriate output of the network;
- converting the resulting representations into a convenient format for the rest of your pipeline.

Do not assume that the final prediction of a pretrained model is necessarily the representation that should be used. Investigate the architecture and determine which output is appropriate for representing an image.

3.3 Understanding your representation

Before continuing, make sure that you understand the representation you have constructed.

Answer the following questions:

1. Which pretrained model did you choose?
2. Was the model pretrained using supervised or self-supervised learning? If it is self-supervised, briefly describe its pretraining objective.
3. Which dataset or collection of data was used to pretrain the model?
4. Which output of the model are you using as the image representation?
5. What is the dimensionality d of your representation?
6. What preprocessing is required by the pretrained model?
7. Do you normalize the representation vectors before using them? Explain your decision.

3.4 Sanity checks

Before using these representations in PhotoMap, perform a few basic checks.

Verify that the representation matrix has the expected shape

$$Z \in \mathbb{R}^{N \times d},$$

where N corresponds exactly to the number of photographs contained in the collection.

Inspect some basic properties of the resulting vectors. For example, you may examine their norms and verify that different photographs do not produce identical representations.

You should also maintain the correspondence between each row of Z and the original photograph:

$$x_i \longleftrightarrow \mathbf{z}_i.$$

This correspondence will be essential when the representations are later used by the web application.

3.5 At the end of this step

At this point, you should have implemented the transformation

$$\boxed{\text{photographs}} \xrightarrow{\text{pretrained encoder}} \boxed{Z \in \mathbb{R}^{N \times d}}.$$

Do **not** reduce these representations to two dimensions and discard the original vectors.

The d -dimensional representations will later be used independently for different components of PhotoMap.

4 Building the Photo Map

You now have one d -dimensional representation for each photograph:

$$Z \in \mathbb{R}^{N \times d}.$$

The first functionality that you will implement is the **Photo Map**.

The objective is to display the complete collection on a two-dimensional space in such a way that the organization of the learned representation can be visually explored.

4.1 From d dimensions to two dimensions

The representations produced by your pretrained encoder cannot generally be displayed directly.

You therefore need to construct a mapping

$$g : \mathbb{R}^d \rightarrow \mathbb{R}^2$$

such that

$$\mathbf{y}_i = g(\mathbf{z}_i), \quad \mathbf{y}_i = \begin{bmatrix} u_i \\ v_i \end{bmatrix}.$$

For the complete collection, this gives

$$Y \in \mathbb{R}^{N \times 2}.$$

The coordinates in Y will determine where each photograph is displayed in the Photo Map.

4.2 Task 2 — Project the representations

Choose and implement an appropriate dimensionality-reduction method.

Possible approaches include, but are not limited to:

- Principal Component Analysis (PCA);
- t-distributed Stochastic Neighbor Embedding (t-SNE);
- Uniform Manifold Approximation and Projection (UMAP);
- another dimensionality-reduction method that you consider appropriate.

You are encouraged to experiment with more than one method before making your final choice.

Your implementation should provide the two-dimensional coordinates required by the existing

`project_photos(...)`

function of the PhotoMap backend.

For each photograph, this function must return the information expected by `app.py`, including its filename and its two-dimensional position.

You may inspect `random_pipeline.py` to understand the expected output format.

4.3 Coordinates expected by PhotoMap

The web interface expects normalized coordinates, with both axes contained approximately in the interval

$$[0, 1].$$

If your dimensionality-reduction method produces arbitrary coordinates, transform each dimension to the required interval before returning the results.

For example, for one coordinate u ,

$$\tilde{u}_i = \frac{u_i - \min_j u_j}{\max_j u_j - \min_j u_j}.$$

The same transformation can be applied independently to the second coordinate.

Once `project_photos(...)` is connected to your implementation, refresh the application and open the **Photo Map** view.

The artificial organization produced by `random_pipeline.py` should now be replaced by the organization obtained from your learned representations.

4.4 Investigating the map

Do not consider the first visualization you obtain to be automatically satisfactory.

Explore the resulting Photo Map and investigate the following questions:

1. Which dimensionality-reduction method did you choose, and why?
2. Which properties of the original representation does this method attempt to preserve?
3. Do photographs that appear visually or semantically related tend to appear close to each other?
4. Can you identify regions of the map corresponding to particular types of photographs?
5. Are there photographs whose position appears surprising? Inspect some examples and try to explain their position.
6. How sensitive is the resulting map to the parameters of your dimensionality-reduction method?
7. If you compare two different projection methods, do they produce the same apparent organization of the collection?

4.5 Be careful with the visualization

The two-dimensional coordinates are a **projection** of the learned representation. They are not the original representation itself.

In general,

$$\mathbf{z}_i \in \mathbb{R}^d \quad \neq \quad \mathbf{y}_i \in \mathbb{R}^2.$$

Reducing a high-dimensional representation to only two dimensions necessarily loses or distorts some information.

Consequently, two photographs appearing very close on the Photo Map are not necessarily the two most similar photographs according to the original representation.

Likewise, photographs that appear relatively far apart in the map may still be close in the original representation space.

Keep both representations:

$$\boxed{Z \in \mathbb{R}^{N \times d}} \quad \text{and} \quad \boxed{Y \in \mathbb{R}^{N \times 2}}.$$

They serve different purposes.

4.6 At the end of this step

Your pipeline should now provide

$$\text{photographs} \longrightarrow Z \in \mathbb{R}^{N \times d} \longrightarrow Y \in \mathbb{R}^{N \times 2},$$

and the **Photo Map** view of the application should display the real photographs according to the two-dimensional projection obtained from your pipeline.

5 Finding Similar Photographs

The Photo Map provides a global visualization of the collection. We now want to support a different interaction: selecting one photograph and retrieving the photographs that are most similar to it.

When a photograph is selected in PhotoMap, the application calls

```
find_neighbours(...)
```

and displays the returned photographs together with their similarity scores.

Your objective is to replace the artificial neighbours generated by `random_pipeline.py` with neighbours computed from the learned representations.

5.1 Nearest-neighbour retrieval

Consider a selected photograph x_q and its representation

$$\mathbf{z}_q \in \mathbb{R}^d.$$

For every other photograph x_i , you need to determine how similar its representation $\mathbf{z}_i$ is to $\mathbf{z}_q$.

The photographs can then be ranked according to this measure and the k most similar photographs returned to the application.

Conceptually,

$$x_q \longrightarrow \mathbf{z}_q \longrightarrow \text{similarity with all } \mathbf{z}_i \longrightarrow k \text{ nearest photographs.}$$

5.2 Task 3 — Implement similarity search

Implement

```
find_neighbours(...)
```

using the representations produced by your pretrained encoder.

You are free to choose the similarity or distance measure used to compare two representations.

Possible choices include, but are not limited to:

- cosine similarity;
- Euclidean distance;
- another similarity or distance measure that you consider appropriate.

The selected photograph itself must not be returned as one of its own neighbours.

You may inspect `random_pipeline.py` to determine the format expected by `app.py`.

Your implementation must return, for each retrieved photograph, its filename and a similarity score that can be displayed by the application.

5.3 Which representation should be compared?

At this point, your pipeline contains two possible representations of each photograph:

$$\mathbf{z}_i \in \mathbb{R}^d,$$

obtained from the pretrained encoder, and

$$\mathbf{y}_i \in \mathbb{R}^2,$$

obtained after dimensionality reduction for the Photo Map.

Before implementing your final similarity search, investigate the following question:

In which space should nearest neighbours be computed?

You may experimentally compare neighbours obtained using the original d -dimensional representations with neighbours obtained using the two-dimensional Photo Map coordinates.

Explain which representation you finally use and justify your choice.

Remember that the two-dimensional representation was introduced primarily to make the collection visually accessible. Dimensionality reduction can modify the distances and neighbourhood relationships present in the original representation space.

5.4 Choosing a similarity measure

Your choice of similarity measure should also be justified.

For example, cosine similarity between two representations is defined as

$$s_{\cos}(\mathbf{z}_i, \mathbf{z}_j) = \frac{\mathbf{z}_i^\top \mathbf{z}_j}{\|\mathbf{z}_i\|_2 \|\mathbf{z}_j\|_2}.$$

Euclidean distance is defined as

$$d_{\text{Euc}}(\mathbf{z}_i, \mathbf{z}_j) = \|\mathbf{z}_i - \mathbf{z}_j\|_2.$$

These two measures do not necessarily produce the same ranking.

Also consider the relationship between your similarity measure and any normalization applied to the representation vectors.

5.5 Test the application

Once your implementation is connected, select several photographs from different regions of the collection.

For each selected photograph, inspect the retrieved neighbours.

Investigate the following questions:

1. Which similarity or distance measure did you choose, and why?
2. Are your representation vectors normalized before computing similarity? What effect does this have?

3. Are the retrieved photographs visually similar to the query?
4. Are they semantically similar even when their visual appearance is different?
5. Can you identify examples where the retrieved neighbours are surprising or incorrect?
6. What properties of the photographs appear to influence the representation most strongly?

Do not limit your analysis to one query photograph. Test several different types of images and investigate both successful and unsuccessful retrievals.

5.6 Similarity scores

The frontend displays a similarity score for each retrieved photograph.

If your chosen measure naturally produces a similarity value, you may use or transform this value into a suitable score for visualization.

If you use a distance measure, remember that a smaller distance represents greater similarity. You should therefore define an appropriate conversion before displaying the value as a similarity score.

Clearly document how the score displayed by PhotoMap is obtained.

5.7 At the end of this step

Selecting a photograph in PhotoMap should now retrieve its neighbours from the learned representation rather than from the artificial grouping of the random pipeline.

Your pipeline now supports two different uses of the same learned representation:

$$\boxed{Z \in \mathbb{R}^{N \times d}} \begin{cases} \longrightarrow \text{2D Photo Map,} \\ \longrightarrow \text{similar-photo retrieval.} \end{cases}$$

The next step will use the same representation once again, this time to automatically organize the complete collection into groups.

6 Automatically Organizing the Collection

The final major functionality of PhotoMap is the automatic organization of the complete photo collection into groups.

Unlike similarity search, which starts from a selected photograph, this task should discover structure in the complete collection without requiring a query image.

The application obtains these groups through

```
discover_groups(...)
```

Your objective is to replace the artificial groups produced by `random_pipeline.py` with groups discovered from the learned representations.

6.1 From representations to groups

You currently have a representation matrix

$$Z \in \mathbb{R}^{N \times d},$$

where each row corresponds to one photograph.

You now want to associate photographs with groups

$$C_1, C_2, \dots, C_K,$$

where each C_k contains photographs whose representations share some structure according to your chosen clustering strategy.

Importantly, the photographs in

`data/photos/`

must still be considered **unlabeled**.

The groups must therefore be discovered from the representations rather than constructed from known semantic categories.

6.2 Task 4 — Discover groups

Implement

`discover_groups(...)`

using an unsupervised clustering method.

Possible approaches include, but are not limited to:

- K -Means;
- agglomerative clustering;
- DBSCAN;
- HDBSCAN;
- another clustering method that you consider appropriate.

You are encouraged to investigate several possibilities before selecting your final approach.

You may inspect `random_pipeline.py` to determine the format expected by `app.py`.

Once implemented, the **Groups** view should display the groups obtained from your clustering algorithm rather than the artificial groups generated by the random pipeline.

6.3 How many groups are there?

For a real personal photo collection, the number of meaningful groups is not known beforehand.

You are therefore **not given the value of K** .

If your chosen clustering method requires a predefined number of clusters, you must propose a strategy for selecting it.

For example, you may investigate criteria such as:

- silhouette score;
- inertia and the elbow method;
- another quantitative or qualitative criterion.

Alternatively, you may choose a clustering method that does not require the number of groups to be specified directly.

Explain how the number of groups produced by your final pipeline is determined.

6.4 Which representation should be clustered?

As in the previous section, you now have access to both

$$Z \in \mathbb{R}^{N \times d},$$

the original learned representations, and

$$Y \in \mathbb{R}^{N \times 2},$$

the coordinates used by the Photo Map.

You must decide which representation should be used for clustering.

Do not assume that clustering the two-dimensional Photo Map is necessarily equivalent to clustering the original representation.

You may compare both approaches experimentally.

Explain your final choice and discuss the possible effect of dimensionality reduction on the discovered groups.

6.5 Investigating the discovered groups

Once the Groups view is functional, inspect the resulting organization.

For each group, examine several photographs and try to understand what properties of the representation may have caused them to be grouped together.

Investigate the following questions:

1. Which clustering method did you choose, and why?
2. Does your method require the number of groups to be specified? If so, how did you select it?
3. Which representation did you cluster: the original d -dimensional embeddings, the two-dimensional projection, or another representation? Why?
4. Are the discovered groups visually coherent?
5. Are some groups semantically coherent even when the photographs do not look immediately similar?
6. Do some groups appear to contain several different concepts?
7. Are there photographs that appear to belong to the wrong group?
8. Are there isolated or unusual photographs that are difficult to assign to any group?

Use concrete examples from the collection to support your observations.

6.6 What does a group represent?

Be careful when interpreting the discovered clusters.

A cluster is a group of photographs that are close according to the representation and clustering strategy that you selected. It is not necessarily equivalent to a human-defined semantic category.

For example, a representation may organize photographs according to properties such as

- semantic content;
- visual appearance;
- shape;
- texture;
- background;
- color;
- composition.

Several of these factors may influence the learned representation at the same time.

The objective is therefore not simply to obtain groups, but also to understand **what structure your representation has discovered**.

6.7 At the end of this step

Your learned representation should now support all three intelligent components of PhotoMap:

$$\boxed{Z \in \mathbb{R}^{N \times d}} \begin{cases} \rightarrow \text{2D Photo Map,} \\ \rightarrow \text{similar-photo retrieval,} \\ \rightarrow \text{automatic grouping.} \end{cases}$$

At this point, the artificial behavior of `random_pipeline.py` should have been completely replaced by your own representation-learning pipeline.

7 Evaluation and Final Analysis

At this stage, PhotoMap should be fully functional using your own representation-learning pipeline. Before concluding, evaluate the complete system and summarize the main choices you made.

7.1 Task 5 — Evaluate your pipeline

A working application does not necessarily imply that the selected methods are appropriate.

Choose at least **one component** of your pipeline and compare it with an alternative. For example:

- two pretrained encoders;
- a self-supervised encoder and a supervised ImageNet baseline;
- two dimensionality-reduction methods;

- two similarity measures;
- two clustering strategies.

You are not expected to explore every possible combination. Instead, formulate one meaningful comparison and support your conclusions with experimental evidence.

Since the photo collection must remain unlabeled, your evaluation should not rely on ground-truth semantic categories.

Depending on your experiment, you may use an appropriate quantitative measure, such as the silhouette score for clustering, together with qualitative inspection of the Photo Map, retrieved neighbours, or discovered groups.

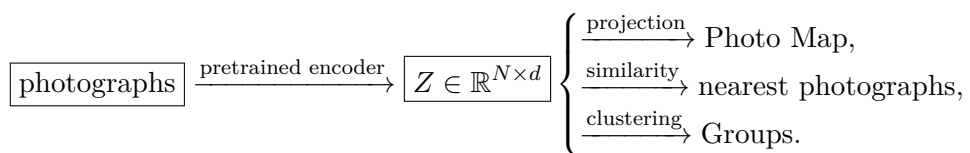
Discuss both successful examples and interesting failure cases.

7.2 Final recap

Briefly summarize your final pipeline:

1. Which pretrained encoder and representation did you use?
2. Which method did you use to construct the Photo Map?
3. How did you retrieve similar photographs?
4. How did you discover groups and determine their number?
5. What did your final comparison reveal?
6. What are the main limitations of your solution?

Your final PhotoMap pipeline can be summarized as



The same pretrained representation is therefore reused for several downstream tasks without training a new model on the PhotoMap collection.